

Monitor UPS

Guia completa de instalacion — Arquitectura MVC con FastAPI

Version 2.0 | 13/04/2026

Descripcion del sistema

Monitor UPS es un sistema de monitoreo de equipos de alimentacion ininterrumpida por red usando el protocolo SNMP. Soporta equipos Eaton (modelos 9E, 9PX, 9SX) y Kaise. Construido con arquitectura MVC usando FastAPI como servidor web, con dashboard accesible desde cualquier navegador de la red local.

Componente	Descripcion
Python 3.12+	Lenguaje de programacion base
FastAPI	Framework web moderno (reemplaza HTTPServer)
Uvicorn	Servidor ASGI que ejecuta FastAPI
Net-SNMP	Herramientas snmpget y snmpwalk para consultar UPS
openpyxl	Generacion de archivos Excel para el historial semanal
Jinja2	Motor de templates HTML (incluido con FastAPI)
NSSM	Servicio Windows para arranque automatico (opcional)

Paso 1 — Instalar Python

Descargar Python 3.12 o superior desde:

<https://www.python.org/downloads>

IMPORTANTE: durante la instalacion marcar la casilla 'Add Python to PATH' antes de hacer clic en Install Now.

Verificar en terminal (CMD o PowerShell):

```
python --version
```

Debe mostrar Python 3.12.x o superior.

Paso 2 — Instalar Net-SNMP

Descargar el instalador desde:

```
https://sourceforge.net/projects/net-snmp/files/net-snmp%20binaries/5.9.1-1/
```

Buscar y descargar: net-snmp-5.9.1-1.x64.exe

Ejecutar con opciones por defecto. Abrir terminal nueva y verificar:

```
snmpget --version
```

Paso 3 — Instalar librerías Python

Abrir terminal y ejecutar:

```
pip install fastapi uvicorn jinja2 python-multipart openpyxl
```

Verificar:

```
python -c "import fastapi; import uvicorn; import openpyxl; print('OK')"
```

Debe responder: OK

Paso 4 — Crear estructura de carpetas MVC

Crear una carpeta para el proyecto, por ejemplo C:\monitor_ups, y ejecutar:

```
mkdir models && mkdir models\snmp && mkdir controllers mkdir templates && mkdir static
```

La estructura final debe quedar así:

```
monitor_ups/ main.py <- Servidor FastAPI y rutas HTTP models/ ups_model.py <-  
Clases UPS, DatosUPS, Alerta config_model.py <- Lectura y escritura de  
ups_config.json historial_model.py <- Generacion de Excel semanales snmp/ base.py  
<- Funcion snmpget() y test_snmp() eaton.py <- Consultas Eaton 9E/9PX y 9SX  
kaise.py <- Consultas Kaise controllers/ ups_controller.py <- Cache, ciclo  
paralelo, consultar_uno() alerta_controller.py <- evaluar_alertas() con umbrales  
config_controller.py <- listar, agregar, editar, eliminar UPS  
historial_controller.py <- leer, listar y descargar Excel templates/ index.html <-  
Dashboard HTML static/ style.css <- Estilos del dashboard app.js <- JavaScript del  
dashboard ups_config.json <- Lista de UPS (auto-generado) historial/ <- Excel  
semanales (auto-generado)
```

Paso 5 — Copiar los archivos del sistema

Copiar todos los archivos Python, HTML, CSS y JS en sus carpetas correspondientes segun la estructura del paso anterior. El archivo ups_config.json y la carpeta historial/ se generan automaticamente al primer arranque.

El archivo ups_config.json contiene la lista de UPS configurados. Si ya existe de una instalacion anterior, copiarlo tambien para no perder la configuracion.

Paso 6 — Configuracion del main.py

El archivo main.py es el punto de entrada de la aplicación. Define todas las rutas HTTP y conecta los controllers con las vistas. Nota importante: en Python 3.14 Jinja2 tiene un bug de compatibilidad. Usar FileResponse en lugar de TemplateResponse:

```
@app.get('/') async def dashboard(): return FileResponse('templates/index.html')
```

En el index.html referenciar los archivos estáticos con rutas directas (no usar url_for de Jinja2):

```
link rel stylesheet href /static/style.css script src /static/app.js
```

Paso 7 — Ejecutar el servidor

Abrir terminal en la carpeta del proyecto y ejecutar:

```
uvicorn main:app --host 0.0.0.0 --port 8080
```

Esperar aproximadamente 15 segundos para la primera consulta a todos los UPS. Luego abrir el navegador en:

```
http://localhost:8080
```

La documentación automática de la API está disponible en:

```
http://localhost:8080/docs
```

Durante el desarrollo usar --reload para que el servidor se reinicie automáticamente al guardar cambios:

```
uvicorn main:app --host 0.0.0.0 --port 8080 --reload
```

Paso 8 — Abrir puerto en el firewall

Para que el dashboard sea accesible desde otras PCs de la red, ejecutar en terminal como Administrador:

```
netsh advfirewall firewall add rule name="Monitor UPS" protocol=TCP dir=in  
localport=8080 action=allow
```

Si responde 'Aceptar' el puerto quedó abierto correctamente.

Para conocer la IP del servidor ejecutar ipconfig y buscar Dirección IPv4. Con esa IP cualquier PC de la red puede acceder en:

```
http://IP_DEL_SERVIDOR:8080
```

Paso 9 — Arranque automático con Windows (recomendado)

Para que el monitor arranque automáticamente al encender el servidor sin necesidad de abrir una terminal, usar NSSM:

1. Descargar NSSM desde: <https://nssm.cc/download>
2. Descomprimir y copiar nssm.exe a C:\Windows\System32\
3. Abrir terminal como Administrador y ejecutar:

```
nssm install MonitorUPS
```

4. Completar los campos en la ventana que aparece:

Campo	Valor
Path	C:\Users\[usuario]\AppData\Local\Programs\Python\Python31X\python.exe
Startup directory	C:\monitor_ups
Arguments	-m uvicorn main:app --host 0.0.0.0 --port 8080
Display name	Monitor UPS

5. Hacer clic en Install Service. El monitor arrancara con Windows.

Umbrales de alerta configurados

Los umbrales se definen en controllers/alerta_controller.py y se pueden modificar sin reiniciar el servidor:

Umbral	Valor	Color	Descripcion
UMBRAL_VOLTAJE_MIN	200V	ROJO	Voltaje entrada por debajo de este valor
UMBRAL_BATERIA_MIN	30%	ROJO	Nivel de bateria por debajo de este valor
UMBRAL_CARGA_MAX	40%	NARANJA	Carga superior al porcentaje de potencia nominal
Estado En bateria	-	ROJO	UPS operando con bateria interna
Estado Falla/Bypass	-	ROJO	UPS en estado de falla o bypass manual
Sin respuesta SNMP	-	ROJO	UPS no responde en la red

Modelos Eaton soportados

El sistema detecta automaticamente el tipo de modelo Eaton y usa los OIDs correctos para cada uno:

Tipo	Modelos	Archivo	Nota
Monofasico	9E6Ki, 9E10Ki, 9PX1500, 9PX3000	snmp/eaton.py	consultar_eaton_9e()
Trifasico	9SX 20KP, 9SX 15KP	snmp/eaton.py	consultar_eaton_9sx() - OIDs y estados diferentes
Kaise	KUKs-RT03	snmp/kaise.py	Autonomia en minutos directo

Potencias nominales Eaton (para calculo de carga %)

El porcentaje de carga de los Eaton se calcula dividiendo los Watts reportados por la potencia nominal del modelo. Definido en snmp/eaton.py:

Modelo (clave)	Potencia nominal
9e6ki	4800 W
9e10ki	8000 W

9px 1500irt2u	1500 W
9sx 20kp	20000 W
9px30000irt2u	30000 W

Checklist de instalacion

#	Paso	Listo
1	Instalar Python 3.12+ marcando Add to PATH	0
2	Verificar: python --version	0
3	Instalar Net-SNMP 5.9.1 x64	0
4	Verificar: snmpget --version en terminal nueva	0
5	Ejecutar: pip install fastapi uvicorn jinja2 python-multipart openpyxl	0
6	Verificar: python -c "import fastapi; print(OK)"	0
7	Crear estructura de carpetas MVC	0
8	Copiar todos los archivos Python, HTML, CSS, JS	0
9	Verificar que index.html usa /static/style.css y /static/app.js	0
10	Ejecutar: uvicorn main:app --host 0.0.0.0 --port 8080	0
11	Verificar dashboard en http://localhost:8080	0
12	Verificar API en http://localhost:8080/docs	0
13	Abrir puerto 8080 en firewall de Windows	0
14	Verificar acceso desde otra PC en la red	0
15	(Opcional) Instalar NSSM para arranque automatico	0